
Hive Project Report ADSAI

Aditya Arsh
SR No. 23812
adityaarsh@iisc.ac.in

Bhadbhade Anuj Sujit
SR No. 23904
anuj Sujit@iisc.ac.in

Jithendra Rao Kasibhatla
SR No. 23646
jithendrarao@iisc.ac.in

Muralidhar Rao M
SR No. 23436
muralidharr@iisc.ac.in

Saksham Agrawal
SR No. 23891
sakshama@iisc.ac.in

Shivey Ravi Guttal
SR No. 23915
shiveyravi@iisc.ac.in

Vijay Bharathram Amaruvi
SR No. 23623
vijayba@iisc.ac.in

Abstract

In this project, we built a second brain system that ingests conversations involving multiple people and performs structured extraction and embedding-based pre-processing and stores them in a query-optimized format that works well with MCP. We implemented this using a combination of LLM models, vector embeddings, and LangGraph for orchestrating tool calling, and packaged everything into a dockerized application complete with a frontend.

1 Introduction

The problem of extracting multi-person conversations is fairly non-trivial since we need to not only isolate the voices but we also need to identify the people and also identify unknown voices. Additionally, simply storing the plain text or doing a naive keyword search is not enough to meaningfully query the data. Hence, we built a pipeline which firstly ingests the audio and does the diarization and identification, this script is then pre-processed and the relational database, vector database and knowledge graph is populated. This is then accessible via MCP which uses these to answer user questions with high accuracy. We discuss each of these parts in more detail in the following sections.

2 Speaker Recognition and Diarization

The first step in the pipeline consists of taking unprocessed audio and passing it through AssemblyAI [1] for speaker recognition, which provides timestamped audio parts grouped by the speaker. Then, we create speaker embeddings by feeding the audio through a SpeechBrain model [2]. With each segmented audio, the audio slice corresponding to it is first taken out, then normalized, resampled, and finally, embedded. The embeddings at the segment level are merged into a weighted mean such that each speaker cluster gets one average embedding.

Then, each cluster embedding is tested against the known speaker embeddings using cosine similarity. If the similarity is above the defined threshold, the cluster is allocated to the respective known speaker, otherwise, it is given an unknown label. The pipeline finally delivers a structured transcript that encompasses the timestamps, deduced speaker identities, and text data. This is then utilized to populate our databases.

3 Relational Database and Vector Database

This is the schema we used for our Relational Database. The design has entities, actions, events, dates, and locations fields separated into their respective tables. The many-to-many linking tables (`entity_events` and `entity_actions`) capture richer relationships such as multiple persons attending the same event or a single person being involved in several actions, etc. The actual conversations themselves are recorded in the `lines` table so that fine details are not lost. And the linking is done appropriately such that they are easily queryable.

In addition to the relational structure, summaries are kept with vector embeddings, and a linking table maps each summary back to the exact lines it abstracts. The embedding model we used is Gemini’s model [3]. This combination of normalized relational data and vector representations is able to maintain exact grounding while allowing flexible semantic search and retrieval. We use NeonDB [4] to manage our database since it is easy to manage and stays persistent.

4 Knowledge Graph

This is the schema we used for our knowledge graph. The graph captures conceptual entities such as Person, Topic, Emotion, Item, and Goal, with typed edges representing relationships like `HAS_OPINION`, `FEELS`, `LIKES`, `KNOWS`, and `HAD_INTENT`. These directed relations allow us to model higher-level semantic links that are difficult to express purely through relational schemas.

We use Neo4j [5] as our graph database because its native property-graph model and Cypher query language make it straightforward to encode, traverse, and query these rich connections.

5 LangGraph

This is the LangGraph [6] workflow used to process each transcript which is generated by the `audio_service`. The graph takes care of populating the RDB, VDB and the Knowledge Graph.

Once the lines are parsed, the graph splits into two parallel branches. The first branch takes care of populating the NeonDB and the second branch takes care of the knowledge graph part.

These branches then combine at the end. This makes the pipeline readable, parallel and debuggable.

6 Docker

We used Docker [7] to containerize the entire system. Each major component runs in its own isolated container, which makes the overall architecture more robust, reproducible, and easy to deploy. The audio processing service, API service, MCP database service, and Streamlit frontend are all defined as separate containers in our `docker-compose.yml`. This separation ensures that each service has its own environment, dependencies, and runtime configuration while still being able to communicate over a shared internal network.

The audio service in particular required an older version of PyTorch due to SpeechBrain’s dependency constraints. Docker allows us to pin that environment without affecting the rest of the project, which continues to use newer libraries. The API service (running FastAPI [8]) then depends on the audio container, the MCP service exposes its own endpoint, and the frontend connects to both through internal service URLs. Using Docker also makes GPU access explicit, simplifies environment management, and guarantees that the system runs consistently across machines. The FastAPI container keeps backend logic isolated and communicates with other services via Docker’s internal network.

Overall, this setup keeps the project modular and stable while avoiding dependency conflicts, and it lets each service evolve independently without breaking the others.

7 MCP

A Model Context Protocol [9] is designed to query and retrieval from the databases. The MCP server is connected to Gemini-2.5 Flash. The tools exposed to the MCP for retrieval are as follows:

1. **search_database**: Performs a unified keyword-based search across multiple relational tables (lines, entities, events, actions, locations). It normalizes the query, executes a SQL UNION to collect matching rows, and returns JSON-serialized results with safe handling of mixed data types.
2. **execute_sql**: Executes user-provided, read-only SQL queries against the relational database. It runs inside a protected environment with validation checks to block write operations, ensures safe serialization of returned rows, and outputs the result as a JSON string.
3. **query_knowledge_graph**: Executes read-only Cypher queries against the Neo4j knowledge graph. It rejects all write operations (e.g., CREATE, DELETE, MERGE), safely serializes Neo4j objects (nodes, relationships, paths), and returns structured graph data as JSON.
4. **semantic_search**: Performs vector-based semantic search using precomputed pgvector embeddings. The input query is embedded using the same model used in embedding the transcript [3], cosine similarity is computed in PostgreSQL, and the top matching summaries (and associated conversation lines) are returned in a size-limited, SSE-safe JSON response.

The query is answered by a RAG mechanism wherein multiple tools are called by the LLM after appending the result from the prior tool call to get a robust output.

The system prompt defines the operational boundaries and retrieval workflow of the AI assistant. It embeds the full relational database schema and the structure of the knowledge graph, enabling the model to issue precise SQL and Cypher queries through specialized tools rather than relying on unsupported assumptions. By enforcing strict JSON-based tool calls, read-only access patterns, and abstraction from implementation details, the prompt ensures that the assistant retrieves information reliably, safely, and deterministically.

8 Frontend

The frontend is made using Streamlit. As this is a Python-based server, integrating backend to the frontend is comparatively easier. The frontend web page has three pages. The first page is for uploading the conversation and speaker samples. The second page is a GPT-style chat page for querying the database. The third page is for viewing the summaries of the conversations.

9 LLM Evaluations

LLMs were evaluated on their ability to extract information from the transcript after processing by SpeechBrain. 10 models were tested, chosen on their size and accessibility to us. The LLMs were evaluated based on several metrics, including adherence to JSON schema, similarity to a golden dataset, and scored by a larger model. We found that Gemini 2.5 Flash, closely followed by OpenAI OSS (20B) were the best models for this task.

10 Tech Stack

Component	Technology / Tool
Audio Processing	Python audio preprocessing (resampling, normalization)
Speaker Recognition & Diarization	AssemblyAI, SpeechBrain, PyTorch (legacy version)
Relational Database	NeonDB (Serverless Postgres)
Vector Database	pgvector (Postgres extension), Gemini Embedding Model (text-embedding-004)
Knowledge Graph	Neo4j
AI Workflow Orchestration	LangGraph
LLMs Used	Gemini 2.5 Flash; evaluated: Gemma 1B/4B, LLaMA 3.2 8B, LLaMA 3.3 70B, Mistral 7B/24B, OpenAI OSS 20B, Deepseek R1 8B, Hermes 7B, Phi 3 8B
MCP Layer	MCP Server + Gemini 2.5 Flash
Backend Services	FastAPI
Frontend	Streamlit
Deployment	Docker, docker-compose

11 Future Directions

These are some potential future directions that can be taken. Right now, the user has to manually set up the environment keys, this should be automated using auth.

Proactive reminders, integration with external calendars and to-do lists can be done to make it a more extensive service. We can also explore generating character sketches for more important (high frequency) characters so that general information about them and their abilities for tasks can be more easily queried. Maybe even giving it agentic capabilities can be explored.

Furthermore, actually making this a wearable will be the main work that will have to be done.

Contributions

The contributions of all the team members are listed below.

- **Aditya Arsh:** MCP-knowledge graph integration, chat interface, bug fixes.
- **Bhadbhade Anuj Sujit:** Knowledge graph schema, population and search, Neo4j integration, co-designed the relational DB schema.
- **Jithendra Rao Kasibhatla:** Refactored notebooks into a Dockerized FastAPI + LangGraph pipeline, co-designed the relational DB schema.
- **Muralidhar Rao M:** MCP design, speech processing pipeline, initial transcript processing and relational DB population.
- **Saksham Agrawal:** Gemini MCP integration, frontend work, final MCP–frontend pipeline and data sync fixes in relational DB.
- **Shivey Ravi Guttal:** Knowledge graph implementation and population, formation and integration of frontend from backend, final pipeline components.
- **Vijay Bharathram Amaruvi:** Transcript processing, populating relational DB, LLM evaluations.

Acknowledgments

Thank you *Prof. Deepak Subramani* for providing us with this opportunity.

References

- [1] AssemblyAI. *AssemblyAI Speech-to-Text and Audio Intelligence API*. 2025. URL: <https://www.assemblyai.com/>.
- [2] Mirco Ravanelli et al. *SpeechBrain: A General-Purpose Speech Toolkit*. 2021. arXiv: 2106.04624 [eess.AS]. URL: <https://arxiv.org/abs/2106.04624>.
- [3] Google DeepMind. *Google Gemini Text Embedding Model: text-embedding-004*. <https://ai.google.dev/models/embedding/text-embedding-004>. 2024.
- [4] Neon Technologies, Inc. *Neon Database: Serverless Postgres with Separation of Storage and Compute*. <https://neon.tech/>. Accessed: 2025-02-17. 2025.
- [5] Neo4j, Inc. *Neo4j Graph Database: Native Graph Storage and Querying*. <https://neo4j.com/>. Accessed: 2025-02-17. 2025.
- [6] LangChain Inc. *LangGraph: A Framework for Building Stateful AI Agents as Graphs*. <https://github.com/langchain-ai/langgraph>. Version 0.1.0. 2025.
- [7] Docker Inc. *Docker: Containerization Platform*. <https://www.docker.com/>. Version 26.1.0. 2025.
- [8] Sebastián Ramirez. *FastAPI*. <https://fastapi.tiangolo.com/>. Accessed: 2025-11-19. 2018.
- [9] *Model Context Protocol (MCP) Documentation*. <https://modelcontextprotocol.io/docs>. Accessed: 2025-11-19. 2025.

A Relevant Images

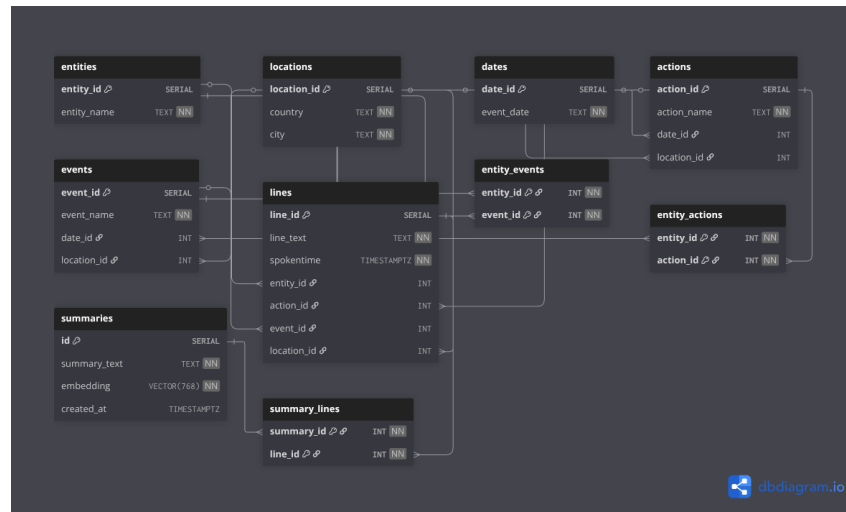


Figure 1: RDB Schema

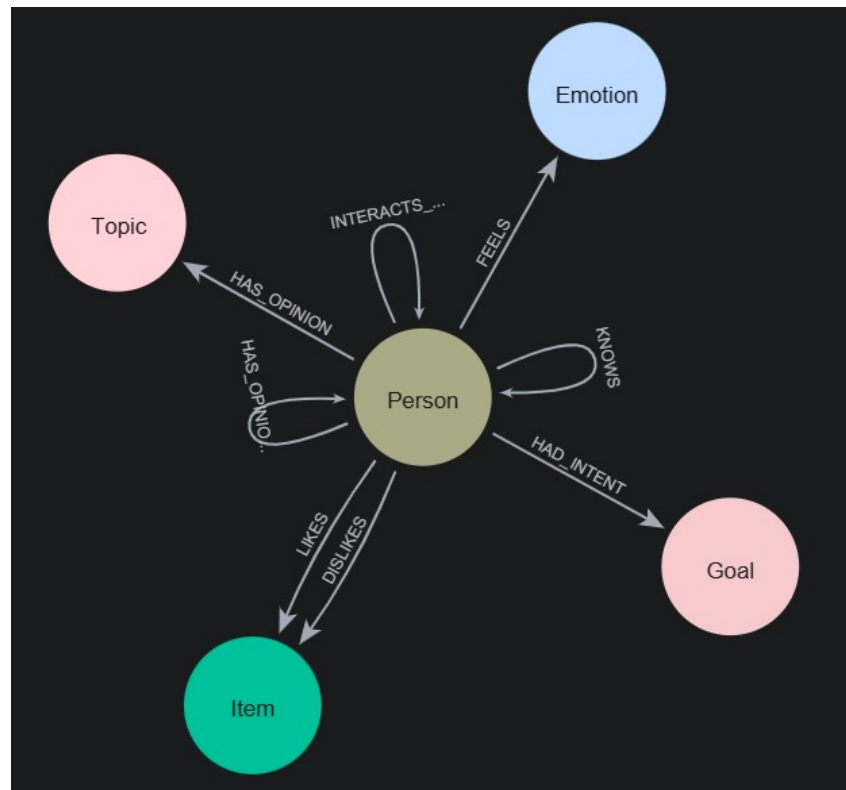


Figure 2: Knowledge Graph Schema

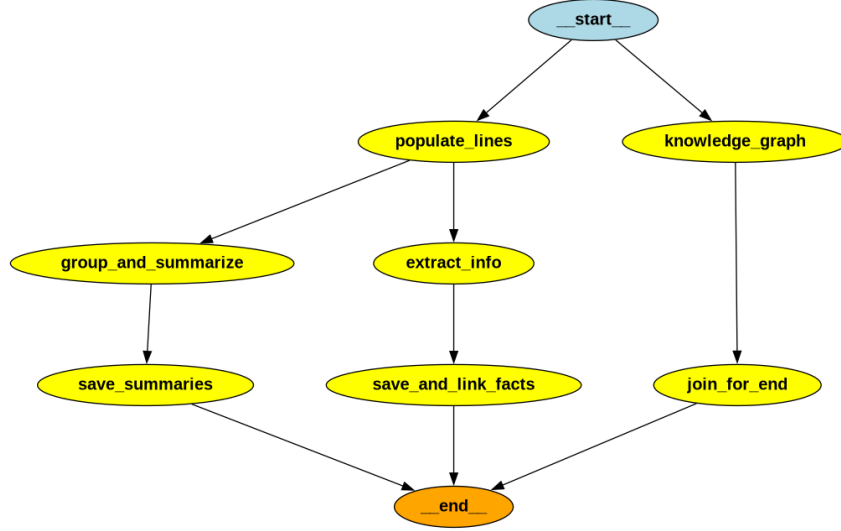


Figure 3: LangGraph

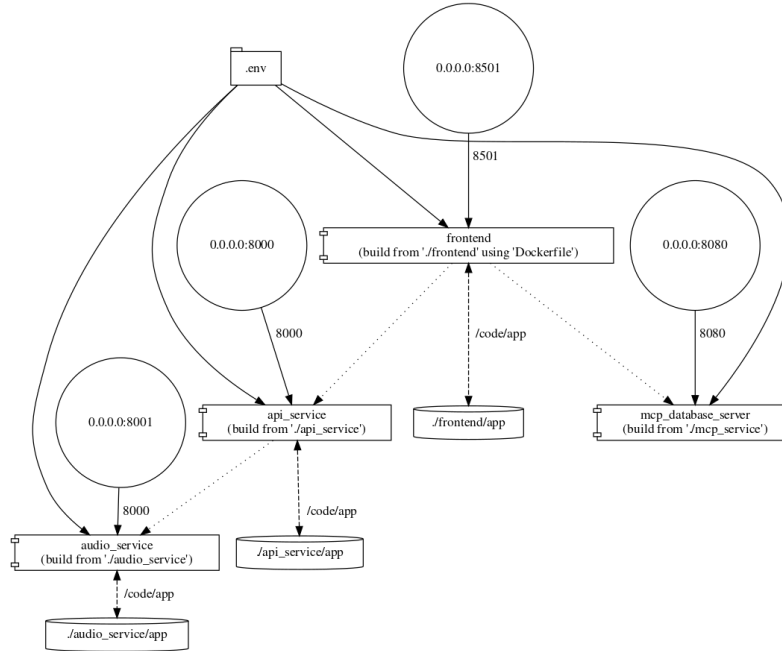


Figure 4: Docker Container Layout

B LLM Evals

B.1 The Models chosen for the task

The models chosen represent a mixture of models of varying sizes, to explore the possibility of running this locally using minimal hardware, and comparing the results as parameter size increases

B.2 Testing Methodology

Firstly, the models were tested on their ability to fill the fields and adhere to the JSON structure.

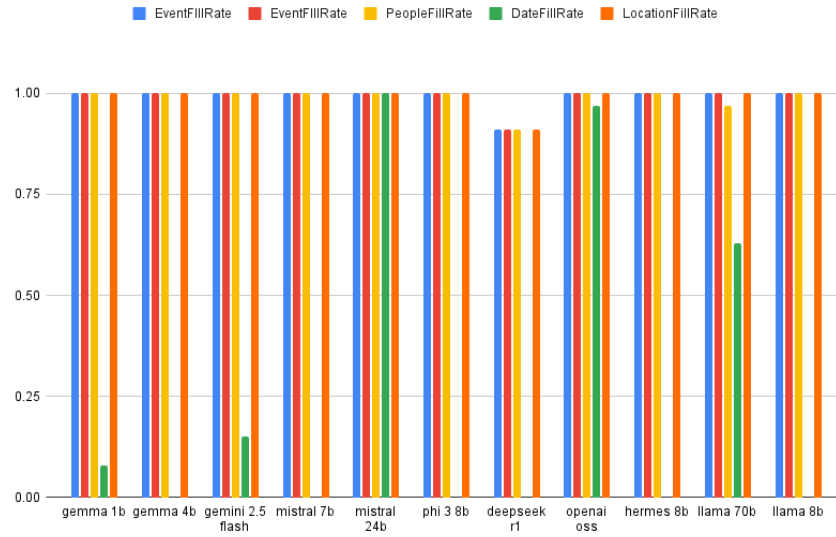
A golden dataset was prepared, consisting of conversation snippets containing events and actions, and "filler" content. The results were then compared to handcrafted JSON outputs for the golden dataset. F1 score, precision, recall were calculated, using semantic similarity and a threshold of 0.8.

The models were then graded on their output by Gemini 2.5 pro on a larger dataset on their accuracy, format, completeness and overall score, and averaged.

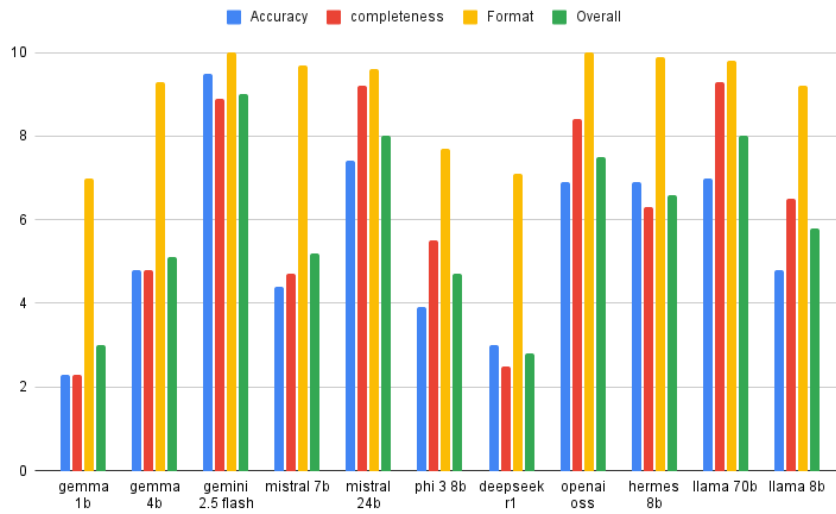
B.3 Results

The mid size and large models performed the best, with the smaller models falling behind. OpenAI OSS 20B and Mistral 24B were the best at filling fields as they were able to infer location and date-time information much better than other models. Gemini 2.5 Flash was rated the best in terms of accuracy and overall score, while OpenAI OSS and LLaMa 70B had the best precision score, while 2.5 Flash, OSS and LLaMa 70B were tied for recall and F1 scores.

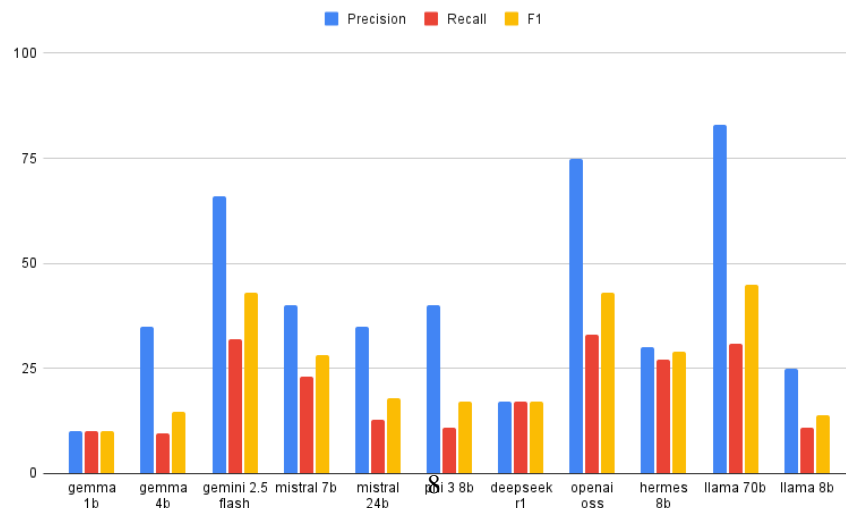
Gemini 2.5 flash was ultimately chosen due to its good performance and speed (and generous token limits), compared to the other models. Refer to this for the relevant tables.



(a) Field-Filling Rate



(b) Gemini 2.5 Pro grade



(c) Precision, Recall, F1